

Scripting Tips and Best Practices

The NGC CLI provides advanced scripting capabilities. You can temporarily override configuration settings with the use of global arguments. You can even save those settings permanently by combining global arguments with the `ngc config set` command.

Tips about Global Arguments

When you specify global arguments for a command, you temporarily override your configuration settings for the duration of the command. If you provide all the required settings through global arguments, then the CLI no longer requires configuration files at all.

The following list identifies important considerations for global arguments:

- Global arguments are not positional and can be specified in any order. The following two commands perform the same action and produce identical results:
- Global arguments always validate the specified values, even if the argument does not affect the command. For example, because the specified organization does not exist, the following command produces an error and does not execute the command:

```
ngc user who --format_type json
ngc user --format_type json who
```

```
ngc user who --org not_existing_org
```

As an example, the output of the `ngc user who` command shows several values from configuration files:

User Id	Name	Email	Org [Roles]	Teams [Roles]
123	Script Master	scripter@nvidia.com	nvidia [REGISTRY_USER]	team_1 [REGISTRY_ADMIN] team_2 [REGISTRY_USER]

By default, the organization and team from the configuration files is used. You can specify global arguments to override those values.

Refer to the following list of global arguments:

- `--debug` Enables debug mode for the current command.
- `--format_type` Updates the output format. Specify *json*, *ascii*, or *csv*.
- `--org` Updates the *org name* used in commands whose help message says that they use the set org.
- `--team` Updates the *team name* used in commands whose help message says that they use the set team.

If you specify an invalid value for a global argument, the CLI exits with a return code of `2` and displays an error message.

Environment Variables

Environment variables provide another way to specify configuration options and credentials. You can use them with scripting to temporarily set them in the shell where you need to run the CLI.

Following environment variables are supported by NGC CLI:

`NGC_CLI_API_KEY`

Specifies the NGC access key (API Key). This is essentially the password to access NGC.

`NGC_CLI_FORMAT_TYPE`

Specifies the output format: JSON, CSV, or ASCII.

`NGC_CLI_HOME`

Specifies the home directory for NGC. The default value is `~/ .ngc`.

`NGC_CLI_ORG`

Specifies the organization name.

`NGC_CLI_TEAM`

Specifies the team name.



Note

Configuration files are stored in an automatically created `.ngc` subdirectory of the path specified in `NGC_CLI_HOME`. If `NGC_CLI_HOME` is not specified, configuration files are stored at `~/ .ngc` on Linux and MacOS. On Windows, the location of the `.ngc` directory depends on other environment variables. The directory is created in the location that is specified by the first variable that is set: `%HOME%`, `%USERPROFILE%`, or `%HOMEDRIVE%%HOMEPATH%`.

Precedence Considerations

- If you specify a value using an environment variable, the value overrides the value loaded from your configuration file.
- If you specify a value using a global argument on the CLI command line, the value overrides any value either from an environment variable or your configuration file.

Example Script

For the following example, imagine that our team wants a daily JSON snapshot of the current user and registry model list. We could set up a cronjob to run the script every day at 7:00 A.M.

1. Install NGC. Refer to [Getting Started with the NGC CLI](#).
2. Create a Bash script like the following code that can run the job:

```
#!/bin/bash
DATE=`date +%m-%d-%yT%H-%M`
# Define my NGC user constants
NGC_CLI_API_KEY="*****"
NGC_CLI_ORG="my_org"

OUTPUT="--format_type json"
OPTS="${OUTPUT}"

# Pre-declare our commands
declare -a commands=("ngc user who" "ngc registry model list")

for command in "${commands[@]}; do
    ${command} ${OPTS}
done
```

3. Cronjob enables you to run specific jobs at certain times during the day. To run the previously created script, edit your crontab file by entering `crontab -e`. Add the following line to run the job at 7:00 A.M. every day:

```
0 7 * * * /path/to/our/script/file
```



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.